Distributed Databases: Q & A

1 System Architecture

Q1: Centralized vs. Distributed Databases

Discuss the relative advantages of centralized and distributed databases.

Solution

Centralized Databases:

- **Simplicity:** Easier to design and maintain as there are no network or synchronization issues.
- **Security:** Centralized control makes it easier to enforce security and integrity constraints.
- **Consistency:** Easier to maintain a single ”source of truth” without complex protocols like 2PC.

Distributed Databases:

- **High Availability:** The failure of one site does not bring down the entire system.
- **Scalability:** Capacity can be increased by adding new sites rather than just upgrading a single server.
- **Performance:** Local data access and parallel query execution significantly reduce response times.
- **Autonomy:** Allows local branches to have control over their own data while still being part of a global system.

Q2: Homogeneous vs. Heterogeneous Systems

What are the fundamental differences between homogeneous and heterogeneous distributed database systems?

Solution

- **Homogeneous Systems:** All sites use identical DBMS software, are aware of each other, and agree to cooperate. Local sites surrender some autonomy to ensure transaction consistency across sites.
- **Heterogeneous Systems:** Sites may use different schemas and different DBMS software. They may not be aware of each other and provide limited facilities for cooperation, making query and transaction processing significantly more difficult.

Q3: LAN vs. WAN Design

How might a distributed database designed for a local-area network differ from one designed for a wide-area network?

Solution

Data transfer on a local-area network (LAN) is much faster than on a wide-area network (WAN). Thus, replication and fragmentation will not increase throughput and speed-up on a LAN as much as in a WAN. However, even in a LAN, replication has its uses in increasing reliability and availability.

Q4: Failure Types

To build a highly available distributed system, you must know what kinds of failures can occur.

a. List possible types of failure in a distributed system.

b. Which items in your list from part (a) are also applicable to a centralized system?

Solution

a. The types of failure that can occur in a distributed system include:

   i. Site failure.

   ii. Disk failure.

   iii. Communication failure, leading to disconnection of one or more sites from the network.

b. The first two failure types (Site failure and Disk failure) can also occur on centralized systems.

Q5: Utility of Replication and Fragmentation

When is it useful to have replication or fragmentation of data? Explain your answer.

Solution

- **Replication is useful when:**
  - **High Availability is required:** If a site fails, others can still serve data.
  - **Queries are read-heavy:** Multiple replicas allow parallel read operations and reduce network traffic by serving data from the nearest site.
- **Fragmentation is useful when:**
  - **Data has clear locality:** For example, a bank branch primarily accesses accounts belonging to its own customers. Horizontal fragmentation allows these tuples to be stored at that specific site, minimizing data transfer.
  - **Security/Privacy is a concern:** Vertical fragmentation can isolate sensitive attributes (like salary) so they are only stored at highly secure sites.
  - **Load Balancing is needed:** Specific shards can be distributed across sites to balance the processing load.

2 Reliability and Storage

Q6: Data Replication

Highlight the advantages and disadvantages of data replication in a distributed environment.

Solution

Advantages:

- **Availability:** If a site fails, data is available at other replicas.
- **Performance (Read):** Queries can be processed in parallel; data is often local to the user.

Disadvantages:

- **Update Overhead:** Every update must be sent to all sites holding a replica.
- **Complexity:** Managing concurrency and consistency across multiple copies is expensive.

Q7: Replication vs. Remote Backup

Explain the difference between data replication in a distributed system and the maintenance of a remote backup site.

Solution

- **Remote Backup Systems:** All transactions are performed at a primary site, and the backup site is kept synchronized via log records. It offers lower cost and execution overhead but lesser availability (only takes over if primary fails). Transaction code only runs at the primary site, and two-phase commit is avoided.
- **Distributed Systems:** Greater availability is offered by having multiple active copies across sites. Transaction code runs at all participating sites, and transactions follow two-phase commit (2PC) to maintain consistency, incurring higher overhead.

3 Fragmentation Techniques

Q8: Horizontal and Vertical Fragmentation

Define horizontal and vertical fragmentation. How is the original relation reconstructed in each case?

Solution

- **Horizontal Fragmentation:** Shards the relation into subsets of tuples using a selection predicate.
  - *Definition:* $r_i = \sigma_{P_i}(r)$
  - *Reconstruction:* $r = r_1 \cup r_2 \cup \cdots \cup r_n$ (Union)
- **Vertical Fragmentation:** Splits the relation schema into subsets of attributes. A common key (or tuple-id) must be kept in each fragment.
  - *Definition:* $r_i = \Pi_{R_i}(r)$
  - *Reconstruction:* $r = r_1 \bowtie r_2 \bowtie \ldots \bowtie r_n$ (Natural Join)

4 Query Optimization

Q9: The Semijoin Strategy

Explain the steps involved in a semijoin strategy to minimize data transmission during a join between relations r_1 (at S_1) and r_2 (at S_2).

Solution

To compute $r_1 \bowtie r_2$ and get the result at S_1:

1. **Project:** Compute $temp_1 = \Pi_{R_1 \cap R_2}(r_1)$ at S_1.
2. **Ship:** Transfer $temp_1$ from S_1 to S_2.
3. **Join:** Compute $temp_2 = r_2 \ltimes temp_1$ at S_2 (selects tuples of r_2 that match).
4. **Ship Result:** Transfer $temp_2$ (only the matching tuples) back to S_1.
5. **Final Join:** Compute $r_1 \bowtie temp_2$ at S_1 to get the final result.

Q10: Semijoin Calculation

Compute $r \ltimes s$ for the following relations using the semi-join operator:

Relation r

A	B	C
1	2	3
4	5	6
1	2	4
5	3	2
8	9	7

Relation s

C	D	E
3	4	5
3	6	8
2	3	2
1	4	1
1	2	3

Solution

The semijoin $r \ltimes s$ returns tuples from r that have a matching value in the common attribute C of relation s.

- Values of C in s are $\{1, 2, 3\}$.
- Tuples in r where $C \in \{1, 2, 3\}$ are $(1, 2, 3)$ and $(5, 3, 2)$.

Result ($r \ltimes s$):

A	B	C
1	2	3
5	3	2

Q10: Symmetry of Semijoin

Is the expression $r_i \ltimes r_j$ necessarily equal to $r_j \ltimes r_i$? Under what conditions does $r_i \ltimes r_j = r_j \ltimes r_i$ hold?

Solution

No, the semijoin operator is **not commutative**.

- $r_i \ltimes r_j$ results in a relation with the schema of r_i (R_i).
- $r_j \ltimes r_i$ results in a relation with the schema of r_j (R_j).

Since R_i and R_j are typically different, the resulting relations cannot be equal.
Conditions for equality: The equality $r_i \ltimes r_j = r_j \ltimes r_i$ holds if and only if:

1. The schemas are identical ($R_i = R_j$).
2. Both relations contain the same set of tuples for the common attributes, or more simply, if $r_i = r_j$.
3. If $R_i = R_j$, then $r_i \ltimes r_j = r_i \cap r_j$, which is equal to $r_j \cap r_i = r_j \ltimes r_i$.

Q11: Processing Strategies for Fragmented Data

Consider a relation `employee (name, address, salary, plant_number)` fragmented horizontally by plant number. Each fragment has two replicas: one at the New York site and one locally at the plant site. Describe a processing strategy for the following queries entered at San Jose:

a. Find all employees at the Boca plant.

b. Find the average salary of all employees.

c. Find the highest-paid employee at Toronto, Edmonton, Vancouver, and Montreal.

d. Find the lowest-paid employee in the company.

Solution

a. Send the query $\Pi_{\text{name}}(\sigma_{\text{plant\_number='Boca'}}(\text{employee}))$ to the Boca plant; Boca sends the result back.

b. Since New York has replicas of all fragments, compute the average salary at the New York site and send the answer to San Jose.

c. Send the "highest-salaried" query to Toronto, Edmonton, Vancouver, and Montreal; compute at those sites and return answers to San Jose.

d. Send the "lowest-salaried" query to New York (which has the complete data) and return the result to San Jose.

Q12: Multi-Relation Query Strategies

Consider relations `employee (name, address, salary, plant_number)` fragmented horizontally by plant and stored locally, and `machine (machine_number, type, plant_number)` stored entirely at Armonk. Describe strategies for:

a. Find all employees at the plant that contains machine number 1130.

b. Find all employees at plants that contain machines whose type is "milling machine".

c. Find all machines at the Almaden plant.

d. Find `employee` $\bowtie$ `machine`.

Solution

a. **Single Site Lookup:** Find the plant number for machine 1130 at Armonk. Then, send a query to that specific plant site to get the employee details.

b. **Semi-Join like Strategy:** At Armonk, find the set of all plant numbers that have milling machines. Send this list to all plant sites. Each site checks if its plant number is in the list; if so, it returns its employees.

c. **Centralized Query:** Simply query the `machine` relation at Armonk with the filter `plant_number = 'Almaden'`.

d. **Distributed Join:** Since `machine` is centralized and `employee` is fragmented, ship the `machine` relation from Armonk to all plant sites. At each site i, compute $employee_i \bowtie machine$ and send the local results to the query initiation site.

Q13: Impact of Site Location on Strategy

For each of the strategies in the previous question (Q12), state how your choice of a strategy depends on:

a. The site at which the query was entered.

b. The site at which the result is desired.

Solution

The choice of strategy is primarily driven by the goal of **minimizing communication costs**, which depends heavily on location:

- **Query Entrance Site:** If the query is entered at **Armonk**, the lookup for machine details is local (zero cost). If entered at a **Plant Site**, we must first communicate with Armonk to find where to route the rest of the query.
- **Result Destination Site:**
  - If the result is desired at **Armonk**, it is often better to ship fragment results there for final processing.
  - If the result is desired at a **specific site X**, the strategy should minimize the total bits sent across the network to Site X. For example, in a join, we might ship the smaller relation to Site X and join it with the larger one locally if Site X happens to hold one of the relations.
- **General Rule:** A "good" strategy always attempts to perform as much selection and projection as possible at the data source before shipping anything to the result site.

5 Transparency and Naming

Q14: Distinguishing Transparency Layers

Explain how the following differ: fragmentation transparency, replication transparency, and location transparency.

Solution

While all three aim to hide distributed complexities from the user, they focus on distinct dimensions of abstraction:

- **Fragmentation Transparency:** Deals with **logical structure**. The user is unaware that a relation has been partitioned into shards. They query the relation as a whole, while the system transparently maps operations to the specific fragments.
- **Replication Transparency:** Deals with **data redundancy**. The user is unaware of the existence of multiple physical copies. They interact with data items as if they were logically unique; the system handles the synchronization of updates and chooses the best replica for read requests.
- **Location Transparency:** Deals with **physical residency**. The user is unaware of which site or server the data is stored on. They use logical names (or aliases) to access data, and the system resolves these to the current physical network address.

Q15: Transparency and Autonomy

Explain the notions of transparency and autonomy. Why are these notions desirable from a human-factors standpoint?

Solution

Notions:

- **Transparency:** The degree to which the system hides the complexities of distribution (fragmentation, replication, and location) from the user. The system appears as a single, centralized database.
- **Autonomy:** The degree to which local sites can manage their own data, schemas, and software independently, without being entirely controlled by a central authority.

Human-Factors Desirability:

- **Reduced Cognitive Load:** Transparency allows users to focus on their logical data needs without worrying about networking or physical storage details. This simplifies query writing and application development.
- **Local Responsibility:** Autonomy empowers local administrators and users to optimize their local site for their specific needs, leading to better accountability, faster local updates, and a sense of ownership over their data.
- **Seamless Integration:** Together, they allow a large, complex organization to function both as individual units and as a unified whole, making the technology easier to adopt and manage by humans in different roles.